13주차 · 11/26(목)

이벤트 처리 기초 및 응용

[온라인 : 강의영상] onclick, addEventListener() 등 활용, 상태와 렌더링

한신대학교 AI·SW계열 · 웹프로그래밍

학습사이트 hs-web.dreamitbiz.com

13주차

학습 목표

1. addEventListener로 사용자 동작에 반응하는 코드를 쓸 수 있다.
2. preventDefault로 폼의 기본 동작을 막을 수 있다.
3. 이벤트 위임으로 나중에 추가될 요소까지 처리할 수 있다.
4. 상태를 바꾸고 화면을 다시 그리는 구조로 코드를 짤 수 있다.
5. 주요 이벤트 종류를 상황에 맞게 고를 수 있다.

13주차

핵심 키워드

addEventListener · click/input/submit · preventDefault · e.target · 이벤트 위임 · 상태와 렌더링

학습사이트의 본문에서 이 낱말들을 찾아 읽으면 이 시간의 뼈대가 잡힙니다

13주차 · 1

이벤트 — 사용자 동작에 반응하기

addEventListener

```
const btn = document.querySelector('#btn');

btn.addEventListener('click', function () {
  console.log('눌렸다');
});

// 화살표 함수로 (더 많이 씁니다)
btn.addEventListener('click', () => {
  console.log('눌렸다');
});

// 이벤트 객체를 받을 수 있습니다
btn.addEventListener('click', (e) => {
  console.log(e.target);        // 실제로 눌린 요소
  console.log(e.type);          // 'click'
});
```

이벤트 — 사용자 동작에 반응하기 (1/2)

이벤트	언제	어디에
click	눌렀을 때	button, a, 아무 요소
input	입력할 때마다	input, textarea
change	값이 확정될 때	select, checkbox, radio
submit	폼을 제출할 때	form (button이 아닙니다)

이벤트 — 사용자 동작에 반응하기 (2/2)

이벤트	언제	어디에
keydown / keyup	키를 누를 때 / 뗄 때	input, document
mouseenter / mouseleave	마우스가 들어올 때 / 나갈 때	아무 요소
scroll	스크롤할 때	window, 스크롤 상자
DOMContentLoaded	HTML을 다 읽었을 때	document

폼이 새로고침되는 것을 막습니다

```
document.querySelector('#myForm').addEventListener('submit', (e) => {
  e.preventDefault();     // 이게 없으면 페이지가 새로고침됩니다

  const name = document.querySelector('#name').value;
  console.log(name);
});

// 링크가 이동하는 것도 막을 수 있습니다
link.addEventListener('click', (e) => {
  e.preventDefault();
});
```

부모 하나에만 걸어 둡니다

```
// ✗ 문제: 이 코드가 돈 뒤에 추가된 li에는 이벤트가 안 걸립니다
document.querySelectorAll('li').forEach(li => {
  li.addEventListener('click', () => console.log('클릭'));
});

// ✓ 부모에 걸고, 실제로 눌린 것이 무엇인지 안에서 판단합니다
document.querySelector('ul').addEventListener('click', (e) => {
  const li = e.target.closest('li');
  if (!li) return;              // li 밖을 눌렀으면 무시
  console.log(li.textContent);
});

// 이벤트가 자식 → 부모로 거슬러 올라가는 성질(버블링)을 이용한 것입니다.
```

13주차 · 2

상태와 렌더링 — 이 수업의 핵심 구조

화면을 조금씩 고치다 보면 금세 어디가 무엇을 바꾸는지 알 수 없게 됩니다.

상태 → 렌더 구조 (1/2)

```
// ① 상태 — 이 앱이 기억해야 할 것 전부
let todos = ['숙제하기', '운동하기'];

// ② 렌더 — 상태를 보고 화면을 만드는 함수. 그리는 곳은 여기 하나뿐
function render() {
  const ul = document.querySelector('#list');
  ul.textContent = '';                          // 통째로 비우고

  todos.forEach((text, i) => {                  // 통째로 다시 그린다
    const li = document.createElement('li');
    li.textContent = text;

    const del = document.createElement('button');
```

> **대신 "값을 바꾸고 → 화면을 다시 그린다" 한 방향으로만 흐르게 만들면, 코드가 늘어나도 헷갈리지 않습니다.**

상태 → 렌더 구조 (2/2)

```
19      del.textContent = '삭제';
20      del.addEventListener('click', () => {
21        todos.splice(i, 1);   // ③ 상태를 바꾸고
22        render();             // ④ 다시 그린다
23      });
24
25      li.append(del);
26      ul.append(li);
27    });
28  }
29
30  render();   // 처음 한 번
```

이 구조가 좋은 이유

정리

1. **화면을 고치는 곳이 render() 하나**
   뿐이라 "어디서 바뀐 거지?"를 찾을 일이 없습니다.
2. **상태만 보면 지금 화면이 어떨지 알 수 있습니다.**
3. **기능을 추가할 때 상태를 바꾸고 render()를 부르면 끝입니다.**
4. **이 구조를 자동으로 해 주는 것이 14주차에 맛볼 React입니다. 지금 손으로 해 보면 React가 왜 필요한지 몸으로 알게 됩니다.**

상태와 렌더링 — 이 수업의 핵심 구조

1. 매번 전부 다시 그리면 느리지 않을까 걱정될 수 있지만, 수십~수백 개 수준에서는 전혀 문제가 되지 않습니다.

2. 먼저 단순하게 만들고, 느려지면 그때 고치는 것이 순서입니다.

13주차

실습

이 시간의 실습 3개 — 학습사이트의 코드 실습장에서 직접 치고 실행합니다

실습 1 · 기본

실습

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

> 카운터를 만드세요. +1, -1, 초기화 버튼 세 개를 두고 숫자를 화면에 표시합니다. 반드시 render() 함수로 그리는 곳을 하나로 모으세요.

힌트

값(count)과 화면은 다른 것입니다. 값을 바꾼 뒤 render()를 불러야 화면이 바뀝니다.

실습 1 모범답안

모범답안

```
document.body.innerHTML = `
  <h1 id="view">0</h1>
  <button id="minus">-1</button>
  <button id="reset">초기화</button>
  <button id="plus">+1</button>
`;

let count = 0;

function render() {
  document.querySelector('#view').textContent = count;
}

document.querySelector('#plus').addEventListener('click', () => { count++; render(); });
document.querySelector('#minus').addEventListener('click', () => { count--; render(); });
document.querySelector('#reset').addEventListener('click', () => { count = 0; render(); });

render();
```

실습 2 · 응용

실습

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

> 입력칸과 버튼을 만들고, 입력한 글자를 목록에 추가하는 기능을 만드세요. 빈 값은 추가되지 않아야 하고, 추가한 뒤 입력칸은 비워야 합니다.

힌트

form의 submit에 preventDefault를 걸고, trim()으로 공백만 입력한 경우도 걸러 냅니다. 사용자...

실습 2 모범답안 (1/2)

모범답안

```
document.body.innerHTML = `
  <form id="f">
    <input id="text" placeholder="할 일">
    <button>추가</button>
  </form>
  <ul id="list"></ul>
`;

const todos = [];

function render() {
  const ul = document.querySelector('#list');
  ul.textContent = '';
  todos.forEach((t) => {
    const li = document.createElement('li');
    li.textContent = t;          // 사용자 입력이므로 textContent
```

실습 2 모범답안 (2/2)

모범답안

```
    ul.append(li);
  });
}

document.querySelector('#f').addEventListener('submit', (e) => {
  e.preventDefault();
  const input = document.querySelector('#text');
  const v = input.value.trim();
  if (!v) return;
  todos.push(v);
  input.value = '';
  render();
});

render();
```

실습 3 · 심화

실습

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

> 탭 전환을 만드세요. 버튼 3개와 내용 3개를 두고, 누른 버튼의 내용만 보이게 하세요. 보이기·숨기기는 hidden 속성으로, 선택된 버튼 표시는 클래스로 하세요.

힌트

data-tab 속성으로 버튼과 내용을 짝지으면 개수가 늘어도 코드를 안 고쳐도 됩니다.

실습 3 모범답안 (1/2)

모범답안

```
document.body.innerHTML = `
  <style>
    .tab.is-active { font-weight: 700; color: #4338ca; }
    [hidden] { display: none !important; }
  </style>
  <button class="tab is-active" data-tab="a">소개</button>
  <button class="tab" data-tab="b">활동</button>
  <button class="tab" data-tab="c">연락</button>
  <section class="panel" data-panel="a"><p>소개 내용</p></section>
  <section class="panel" data-panel="b" hidden><p>활동 내용</p></section>
  <section class="panel" data-panel="c" hidden><p>연락 내용</p></section>
`;
```

실습 3 모범답안 (2/2)

모범답안

```
const tabs = document.querySelectorAll('.tab');
const panels = document.querySelectorAll('.panel');

tabs.forEach((tab) => {
  tab.addEventListener('click', () => {
    const key = tab.dataset.tab;
    tabs.forEach((t) => t.classList.toggle('is-active', t === tab));
    panels.forEach((p) => { p.hidden = p.dataset.panel !== key; });
  });
});
```

13주차 마무리

13주차 마무리

이벤트 처리 기초 및 응용 — 이 시간에 다룬 것을 학습사이트에서 다시 확인합니다

- [ ] 학습사이트 주차 페이지의 절마다 「이해했어요 · 실습했어요 · 어려워요」 표시
- [ ] 실습 3개를 실습장에서 「확인」 까지 — 「실습했어요」 체크
- [ ] 퀴즈 11 「퀴즈 11 — 이벤트 처리 기초와 응용」 응시 (성적 미반영 · 학습 확인용)
- [ ] 막히면 「질문있어요」 또는 패들릿에 — 30분 넘게 혼자 막혀 있지 않기

다음 시간

14주차 · 종합 정리 및 과제 평가